COMICCRAFT

AI Comic Story Creator using Gemini Models

PROJECT DOCUMENTATION

FastAPI • Jinja2 • Gemini API • Image Generation • PDF Export

A technical report based on the uploaded ComicCraft source-code archive and the supplied reference documentation.

Project scope

A web application that accepts a story idea and character preferences, builds a five-panel comic experience, displays panel content, and exports the result to a downloadable PDF.

Documentation note: visual examples in this report are reused from the supplied reference PDF. They are reference illustrations and are not claimed to be screenshots captured from a fresh run of the uploaded code.

Contents

Section	Page
1. Project overview	3
2. Scenarios and requirements	3
3. Technical architecture	4
4. Project structure	5
5. Environment setup	6
6. Core service modules	8–9
7. FastAPI routes and request schema	10
8. Frontend and templates	12
9. PDF export workflow	13
10. Reference application visuals	14–15
11. Testing and verification	16
12. Configuration and security	17
13. Limitations and future improvements	17
14. Conclusion	18

Source basis

The implementation details in this report are taken from the uploaded ComicCraft_Project.zip, including app/main.py, app/config.py, app/routes.py, app/schemas.py, service modules, templates, requirements.txt, README.md, and tests/test_api.py. The earlier 25-page PDF is used as the requested layout and visual reference. Where the code does not implement a behavior described by the reference PDF, this report describes the code as supplied rather than assuming the feature exists.

1. Project Overview

ComicCraft is a browser-based comic creation application. A user supplies a story prompt, main character name, setting, tone, and preferred art style. The backend processes these values, produces a sequence of comic panels, generates or simulates panel illustrations depending on configuration, displays the result in a preview template, and creates a PDF export.

Project objectives

- Collect creative preferences through a simple HTML form.
- Create a five-panel outline with a title, scene description, and image prompt for each panel.
- Add narration, dialogue, and captions to each panel.
- Generate panel illustrations through demo mode, Hugging Face inference, or a local Diffusers pipeline.
- Display the generated panel data in a Jinja2 template and export it as a multi-page PDF.
- Expose JSON endpoints and interactive API documentation through FastAPI.

Key implementation note

The uploaded source defaults to DEMO_MODE=true. In this mode the outline and story are generated from local deterministic templates and the illustrations are simple labeled images. Real Gemini calls and model-backed image generation are conditional on configuration and credentials.

2. Scenarios and Requirements

Scenario A — Create a comic

A user enters “A brave fox exploring an enchanted forest”, names the character Luna, selects a setting, tone, and art style, and submits the form. The /generate route passes the validated values into the service pipeline and renders comic_preview.html with the panel layout and PDF URL.

Scenario B — Change the mood or art style

The user changes tone or art style in the input form and submits again. These fields are included in the outline and image prompts. In real-model mode they influence model instructions; in demo mode they influence the deterministic sample text and image label.

Scenario C — Export the comic

The exporter creates a timestamped PDF in static/exports. The preview page displays a download link pointing to that PDF. The current route implementation also defines /export-success as a separate page, but the form-generation route does not automatically redirect to it.

Functional requirements

- Homepage form for story prompt, character, setting, tone, and art style.
- POST /generate for browser form submissions.
- POST /generate-comic/json for JSON clients.
- POST /test-image for testing illustration generation.
- GET /docs for interactive FastAPI API documentation.
- Downloadable PDF containing each panel image and narrative text.

3. Technical Architecture

The project is divided into a presentation layer, a FastAPI application layer, service modules, and generated static files.

Layer	Implementation	Responsibility
Presentation	HTML + Jinja2 + CSS	Collect user inputs and display the comic preview.
API / routing	FastAPI (app/main.py, app/routes.py)	Serve pages, validate requests, orchestrate the pipeline.
Validation	Pydantic (app/schemas.py)	Validate JSON payload fields and limits.
Text generation	Gemini service modules	Create a structured outline and panel narration/dialogue.
Image generation	Pillow / Hugging Face / Diffusers	Create a demo image or model-generated illustration.
Layout + export	layout_builder.py + exporters.py	Pass panels through layout stage and build a PDF.
Static storage	static/panels, static/exports	Store generated images and PDF output.

Request flow

Browser form or JSON client → FastAPI route → PromptRequest (for JSON) → generate_outline() → generate_story() → generate_image() for each panel → build_comic_layout() → save_pdf() → preview/JSON response.

External AI integration

The source imports Google Gen AI only when real text generation is enabled. Hugging Face inference and Diffusers are selected through IMAGE_PROVIDER. The default demo mode avoids external model calls and creates local example content.

4. Project Structure

```
ComicCraft/
■■■ app/
■   ■■■ __init__.py
■   ■■■ main.py
■   ■■■ config.py
■   ■■■ routes.py
■   ■■■ schemas.py
■   ■■■ services/
■       ■■■ __init__.py
■       ■■■ gemini_flash.py
■       ■■■ gemini_pro.py
■       ■■■ image_generator.py
■       ■■■ layout_builder.py
■       ■■■ exporters.py
■■■ templates/
■   ■■■ index.html
■   ■■■ comic_preview.html
■   ■■■ export_success.html
■■■ static/
■   ■■■ css/style.css
■   ■■■ panels/
■   ■■■ exports/
■■■ tests/test_api.py
■■■ .env.example
■■■ requirements.txt
■■■ README.md
■■■ .gitignore
```

Main files

- app/main.py — creates the FastAPI instance, mounts /static, and includes the router.
- app/config.py — reads model keys, model names, demo mode, and panel count from environment settings.
- app/routes.py — defines page, generation, JSON, image-test, and export-success routes.
- app/schemas.py — validates the request payload.
- app/services/gemini_flash.py — creates the structured panel outline.
- app/services/gemini_pro.py — expands outline panels with story text.
- app/services/image_generator.py — creates a demo or model-backed illustration.
- app/services/layout_builder.py — currently returns the panels as supplied.
- app/services/exporters.py — creates a timestamped multi-page PDF.

5. Environment Setup and Running

Prerequisites

- Python 3.10+ recommended; use a version compatible with the installed dependencies.
- PowerShell on Windows or a terminal on macOS/Linux.
- Internet access for installing dependencies and calling external model services.
- API credentials are optional for demo mode, but required for configured live services.

Windows PowerShell setup

```
# ComicCraft - AI Comic Story Creator using Gemini Models

This implementation follows the supplied project document: FastAPI + Jinja2 frontend, Gemini for
    structured comic outline/story generation, Stable Diffusion-compatible image generation, five
    panels, and FPDF PDF export.

## Fast setup (Windows PowerShell)
```powershell
cd ComicCraft
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
copy .env.example .env
uvicorn app.main:app --reload

Open http://127.0.0.1:8000 and API docs at http://127.0.0.1:8000/docs.

### First test without API keys
Keep `DEMO_MODE=true` in `.env`. The application will generate deterministic demo illustrations and
    a real PDF, allowing you to verify the entire pipeline.

### Real Gemini + image generation
Set `DEMO_MODE=false`, add `GEMINI_API_KEY`, and set `IMAGE_PROVIDER=huggingface` plus `HF_API_KEY`.
    The image model is Stable Diffusion XL by default. For local Diffusers, set
    `IMAGE_PROVIDER=diffusers`; this requires substantial model download/storage and is much better
    with a compatible GPU.

## Test
```

If you are starting from the extracted project folder, use these commands:

```
cd ComicCraft
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
copy .env.example .env
uvicorn app.main:app --reload
```

Open the application at **http://127.0.0.1:8000**. Open **http://127.0.0.1:8000/docs** to inspect and test API routes. Keep DEMO_MODE=true for the first local run.
```

Environment variables

Variable	Purpose / supplied default
GEMINI_API_KEY	Gemini API key; empty in .env.example
HF_API_KEY	Hugging Face token; empty in .env.example
GEMINI_MODEL	Outline model; gemini-2.5-flash
GEMINI_STORY_MODEL	Story model; gemini-2.5-flash
IMAGE_PROVIDER	huggingface by default; can be demo or diffusers
IMAGE_MODEL	stabilityai/stable-diffusion-xl-base-1.0
DEMO_MODE	true by default
PANEL_COUNT	5 by default

6. Core Service Modules

6.1 Structured outline — gemini_flash.py

The module reads settings. If demo mode is enabled or no Gemini API key is configured, it returns a five-item sample outline. Otherwise, it calls the configured Gemini model with a JSON-only instruction and verifies the returned panel count.

```
import json
from app.config import get_settings
def demo(req):
    names=[('The Call','discovers a mysterious clue'),('Into the Unknown','follows the clue'),('The
        Challenge','faces a surprising obstacle'),('The Turning Point','finds a clever
        solution'),('A New Beginning','makes a hopeful discovery')]
    return [{'panel_number':i+1,'title':t,'scene_description':f'{req.character_name} {d} in
        {req.setting}.','image_prompt':f'comic book panel, {req.art_style}, {req.tone} mood,
        {req.character_name}, {req.setting}, {d}, cinematic, expressive, detailed'} for i,(t,d) in
        enumerate(names)]
def generate_outline(req):
    s=get_settings()
    if s.demo_mode or not s.gemini_api_key:return demo(req)
    from google import genai
    c=genai.Client(api_key=s.gemini_api_key)
    p=f'''Create exactly {s.panel_count} coherent comic panels. Story: {req.story_prompt}.
        Character: {req.character_name}. Setting: {req.setting}. Tone: {req.tone}. Art style:
        {req.art_style}. Return ONLY JSON
        {{"panels":[{{"panel_number":1,"title":"","scene_description":"","image_prompt":""}}]}}.'''
    r=c.models.generate_content(model=s.gemini_model,contents=p,config={'response_mime_type':'applic
        ation/json'})
    panels=json.loads(r.text)['panels']
    if len(panels)!=s.panel_count:raise ValueError('Unexpected panel count from Gemini')
    return panels
```

Output fields include panel_number, title, scene_description, and image_prompt. These fields are consumed by later services.

6.2 Story narration — gemini_pro.py

This module adds narration, dialogue, and captions to the outline. In demo mode, it supplies deterministic sample text; in live mode, it calls Gemini and parses a JSON response. The configuration names the story model separately, although the supplied .env.example uses the same Gemini Flash model for both.

```
import json
from app.config import get_settings
def generate_story(outline,req):
    s=get_settings()
    if s.demo_mode or not s.gemini_api_key:
        return [{**p,'narration':f'{req.character_name} moves through {req.setting}, guided by a
            strange clue.','dialogue':f'{req.character_name}: "I will discover the
            truth!"','caption':f'{req.tone.title()} moment in {req.setting}.'} for p in outline]
    from google import genai
    c=genai.Client(api_key=s.gemini_api_key)
    p=f'''Expand this outline into exactly {len(outline)} comic panels. Character
        {req.character_name}, tone {req.tone}. Return ONLY JSON {{"panels":[{{"panel_number":1,"titl
        e":"","scene_description":"","image_prompt":"","narration":"","dialogue":"","caption":""}}]}
        }. Outline: {json.dumps(outline)}'''
    r=c.models.generate_content(model=s.gemini_story_model,contents=p,config={'response_mime_type':'
        application/json'})
    return json.loads(r.text)['panels']
```

6.3 Image generation — image_generator.py

The image generator writes each panel to static/panels/panel_N.png. Its provider behavior is selected using IMAGE_PROVIDER. Demo mode creates a labeled placeholder image using Pillow; Hugging Face uses InferenceClient; Diffusers loads a StableDiffusionPipeline locally.

```
from pathlib import Path
from PIL import Image,ImageDraw
from app.config import get_settings
ROOT=Path(__file__).resolve().parents[2]; OUT=ROOT/'static'/'panels';
    OUT.mkdir(parents=True,exist_ok=True)
def generate_image(prompt,panel_number):
    s=get_settings(); path=OUT/f'panel_{panel_number}.png'; provider=s.image_provider.lower()
    if s.demo_mode or provider=='demo':
        im=Image.new('RGB',(1024,640),'white'); d=ImageDraw.Draw(im);
            d.rectangle((20,20,1004,620),outline='black',width=6);
            d.text((50,70),'COMICCRAFT',fill='black'); d.text((50,160),prompt[:180],fill='black');
            d.text((50,540),'Demo image - enable Hugging Face or Diffusers for AI
            art.',fill='black'); im.save(path); return f'/static/panels/panel_{panel_number}.png'
    if provider=='huggingface':
        from huggingface_hub import InferenceClient
        if not s.hf_api_key: raise RuntimeError('HF_API_KEY is required')
        img=InferenceClient(provider='hf-
            inference',api_key=s.hf_api_key).text_to_image(prompt,model=s.image_model);
            img.save(path); return f'/static/panels/panel_{panel_number}.png'
    if provider=='diffusers':
        import torch
        from diffusers import StableDiffusionPipeline
        dev='cuda' if torch.cuda.is_available() else 'cpu'; dtype=torch.float16 if dev=='cuda' else
            torch.float32
        pipe=StableDiffusionPipeline.from_pretrained(s.image_model,torch_dtype=dtype).to(dev);
            pipe(prompt,num_inference_steps=20).images[0].save(path); return
            f'/static/panels/panel_{panel_number}.png'
    raise ValueError('Unsupported IMAGE_PROVIDER')
```

6.4 Layout building — layout_builder.py

The supplied layout builder is intentionally minimal: it returns the panel list unchanged. This means the panel dictionaries produced by the story service are used directly by the preview and PDF exporter.

```
def build_comic_layout(panels): return panels
```

6.5 PDF export — exporters.py

The exporter creates a timestamped file under static/exports. It starts a new PDF page for each panel and writes the panel heading, illustration, scene description, caption, narration, and dialogue. The function returns a URL path that the frontend can use as a download link.

```
from pathlib import Path
from datetime import datetime
from fpdf import FPDF
ROOT=Path(__file__).resolve().parents[2]; OUT=ROOT/'static'/'exports';
    OUT.mkdir(parents=True,exist_ok=True)
def save_pdf(layout):
    path=OUT/f'comic_{datetime.now().strftime("%Y%m%d_%H%M%S")}.pdf'; pdf=FPDF();
        pdf.set_auto_page_break(True,15)
    for p in layout:
        pdf.add_page(); pdf.set_font('Helvetica','B',18); pdf.multi_cell(0,10,f"Panel
            {p['panel_number']}: {p['title']}");
            img=ROOT/p['image_path'].lstrip('/').replace('static/','static/');
            pdf.image(str(img),w=175); pdf.set_font('Helvetica','I',11);
            pdf.multi_cell(0,7,p['scene_description']); pdf.set_font('Helvetica','',11);
            pdf.multi_cell(0,7,'Caption: '+p.get('caption','')); pdf.multi_cell(0,7,'Narration:
            '+p.get('narration','')); pdf.multi_cell(0,7,'Dialogue: '+p.get('dialogue',''))
    pdf.output(str(path)); return f'/static/exports/{path.name}'
```

7. FastAPI Routes and Request Schema

The FastAPI application is created in app/main.py. It mounts the project static directory at /static and registers the router from app/routes.py. The main routes are summarized below.

Method	Path	Purpose
GET	/	Render index.html form.
POST	/generate	Accept form fields, run generation pipeline, render comic_preview.html.
POST	/generate-comic/json	Accept a PromptRequest JSON body and return panel layout and PDF URL.
POST	/test-image	Generate a single test image from a prompt.
GET	/export-success	Render export_success.html.
GET	/docs	FastAPI interactive API documentation.

Request validation — schemas.py

PromptRequest validates five required fields. story_prompt must contain 5–2000 characters; character_name must contain 1–100; setting 1–100; tone 1–50; and art_style 1–80. Invalid JSON requests are rejected by FastAPI/Pydantic with a 422 response.

```
from pydantic import BaseModel,Field
class PromptRequest(BaseModel):
    story_prompt:str=Field(...,min_length=5,max_length=2000)
    character_name:str=Field(...,min_length=1,max_length=100)
    setting:str=Field(...,min_length=1,max_length=100)
    tone:str=Field(...,min_length=1,max_length=50)
    art_style:str=Field(...,min_length=1,max_length=80)
```

Route orchestration

```
from fastapi import APIRouter,Request,Form,HTTPException
from fastapi.templating import Jinja2Templates
from app.schemas import PromptRequest
from app.services.gemini_flash import generate_outline
from app.services.gemini_pro import generate_story
from app.services.image_generator import generate_image
from app.services.layout_builder import build_comic_layout
from app.services.exporters import save_pdf
from pathlib import Path
router=APIRouter();
    templates=Jinja2Templates(directory=str(Path(__file__).resolve().parents[1]/'templates'))
def run(req):
    panels=generate_story(generate_outline(req),req)
    for p in panels:p['image_path']=generate_image(p['image_prompt'],p['panel_number'])
    layout=build_comic_layout(panels); return layout,save_pdf(layout)
@router.get('/')
async def home(request:Request):return templates.TemplateResponse('index.html',{'request':request})
@router.post('/generate')
async def generate(request:Request,story_prompt:str=Form(...),character_name:str=Form(...),setting:s
    tr=Form(...),tone:str=Form(...),art_style:str=Form(...)):
    try:
        layout,pdf=run(PromptRequest(story_prompt=story_prompt,character_name=character_name,setting
            =setting,tone=tone,art_style=art_style)); return templates.TemplateResponse('comic_previ
           ew.html',{'request':request,'layout':layout,'pdf_url':pdf})
    except Exception as e: raise HTTPException(500,str(e))
@router.post('/generate-comic/json')
async def generate_json(req:PromptRequest):
    try:
        layout,pdf=run(req); return {'title':f'{req.character_name} Comic','panels':layout,'pdf_url':pdf}
    except Exception as e: raise HTTPException(500,str(e))
@router.post('/test-image')
async def test_image(prompt:str=Form(...)): return {'success':True,'image_path':generate_image(prompt,0)}
@router.get('/export-success')
async def export_success(request:Request):return
    templates.TemplateResponse('export_success.html',{'request':request})
```

The shared run() function coordinates outline generation, story expansion, image creation for every panel, layout construction, and PDF export. The form route returns the preview template; the JSON route returns a title, panels, and pdf_url.

8. Frontend and Templates

The frontend is implemented with HTML templates rendered by Jinja2 and a shared CSS file.

index.html — input page

The homepage contains a story prompt textarea, character-name input, and dropdowns for setting, tone, and art style. Its form posts to /generate. Default sample values allow a user to test the workflow quickly in demo mode.

```
<!doctype html><html><head><title>ComicCraft</title><link rel="stylesheet"
    href="/static/css/style.css"></head><body><header><b>GENERATIVE AI
    PROJECT</b><h1>ComicCraft</h1><p>Turn your imagination into a five-panel
    comic.</p></header><main><section class="card"><h2>Create Your Comic</h2><form
    action="/generate" method="post"><label>Story Prompt</label><textarea name="story_prompt"
    required>A brave fox exploring an enchanted forest</textarea><label>Main Character
    Name</label><input name="character_name" value="Luna" required><label>Setting</label><select
    name="setting"><option>Enchanted Forest</option><option>School</option><option>Future
    City</option><option>Outer Space</option></select><label>Story Tone</label><select
    name="tone"><option>Funny</option><option>Dramatic</option><option>Light-
    hearted</option><option>Adventurous</option></select><label>Art Style</label><select
    name="art_style"><option>Comic Book</option><option>Anime</option><option>Pixel
    Art</option><option>Realistic</option></select><button>■ Generate My
    Comic</button></form></section></main></body></html>
```

comic_preview.html — results page

The preview loops through the layout list. Each panel shows its title, image, scene description, caption, narration, and dialogue. A download link points to pdf_url, and a second link returns the user to the homepage.

```
<!doctype html><html><head><title>ComicCraft Preview</title><link rel="stylesheet"
    href="/static/css/style.css"></head><body><main><h1>Comic Preview</h1><p><a class="button"
    href="{{pdf_url}}" download>Download Your Comic as PDF</a></p>{% for p in layout %}<article
    class="card"><h2>Panel {{p.panel_number}}: {{p.title}}</h2><img
    src="{{p.image_path}}"><p><i>{{p.scene_description}}</i></p><p><b>Caption:</b>
    {{p.caption}}</p><p><b>Narration:</b> {{p.narration}}</p><p><b>Dialogue:</b>
    {{p.dialogue}}</p></article>{% endfor %}<a class="button" href="/">Create Another
    Comic</a></main></body></html>
```

export_success.html — confirmation page

This template provides a simple “Your Comic Is Ready!” message and a link back to the homepage. In the current source, it is available through GET /export-success; it is not automatically used as a redirect after PDF creation.

```
<!doctype html><html><body><h1>Your Comic Is Ready!</h1><a href="/">Go Create Another
    Comic</a></body></html>
```

style.css — visual design

The stylesheet uses a neutral page background, dark header, centered content area, white rounded cards, consistent form fields, and button styling. Images are scaled to fit the preview card.

```
body{font-family:Arial,sans-serif;background:#f4f5f8;color:#222;margin:0}header{padding:50px
    8%;background:#20232a;color:#fff}header h1{font-size:50px;margin:12px 0}main{max-
    width:950px;margin:30px auto;padding:0 18px}.card{background:#fff;padding:28px;border-
    radius:16px;margin:22px 0;box-shadow:0 6px 25px #0001}label{display:block;font-
    weight:bold;margin:16px 0 7px}input,textarea,select{width:100%;padding:12px;border:1px solid
    #ccc;border-radius:8px;box-sizing:border-box}textarea{min-
    height:120px}button,.button{display:inline-block;background:#20232a;color:#fff;padding:13px
    18px;border:0;border-radius:8px;text-decoration:none;margin-top:20px;cursor:pointer}.card
    img{width:100%;border-radius:10px;max-height:650px;object-fit:contain;background:#eee}
```

9. PDF Export Workflow

The export process is part of the generation pipeline rather than a separate frontend-only action. After panels are assembled, save_pdf() writes one page per panel using FPDF2. The preview's "Download Your Comic as PDF" anchor opens the resulting file with a download attribute.

Export sequence

- Create output folder static/exports if it does not already exist.
- Create a filename such as comic_YYYYMMDD_HHMMSS.pdf.
- For every panel, add a page and write its panel number and title.
- Place the image on the page, followed by scene description, caption, narration, and dialogue.
- Save the PDF and return a /static/exports/... URL to the route.

Generated output locations

Illustrations are stored under static/panels/. Exported PDFs are stored under static/exports/. Both directories are included in the ZIP with .gitkeep placeholders; actual generated output files appear after the application is run.

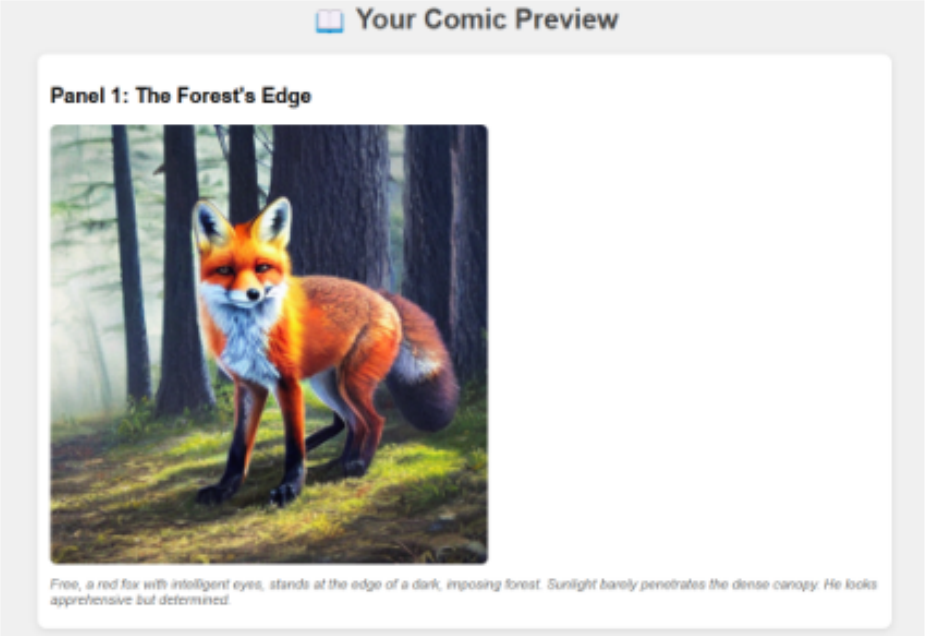
Current implementation note

The /export-success route is present but the normal form flow renders the preview directly. The current PDF link is the direct download path. This report documents that actual behavior rather than describing an automatic redirect that is not implemented in the supplied routes.

10. Reference Application Visuals

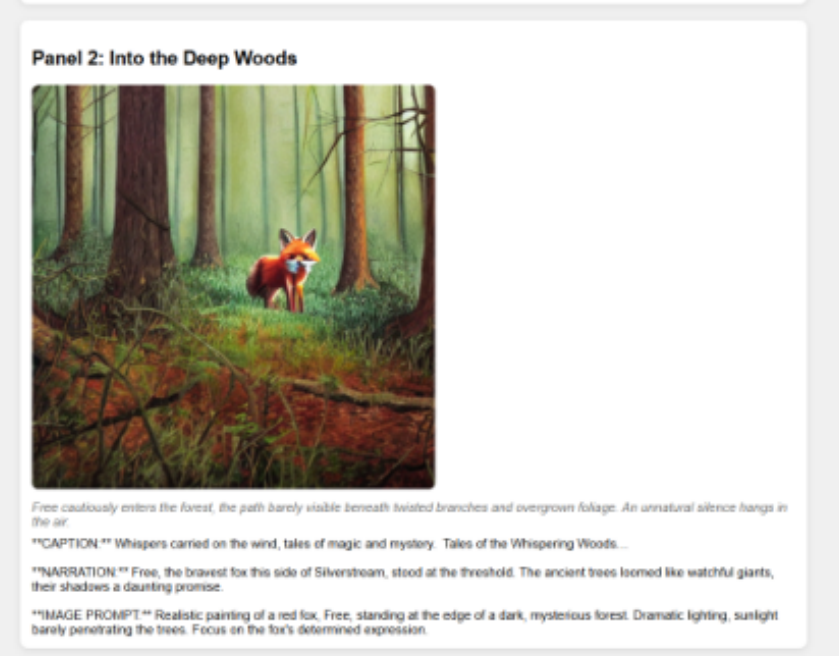
The following visuals are cropped from the original 25-page ComicCraft reference PDF supplied in this conversation. They are included because separate current-run screenshots were not present in the project ZIP. They illustrate the intended application experience but should not be treated as proof that the uploaded source currently renders identical screens.

Comic panel / preview example



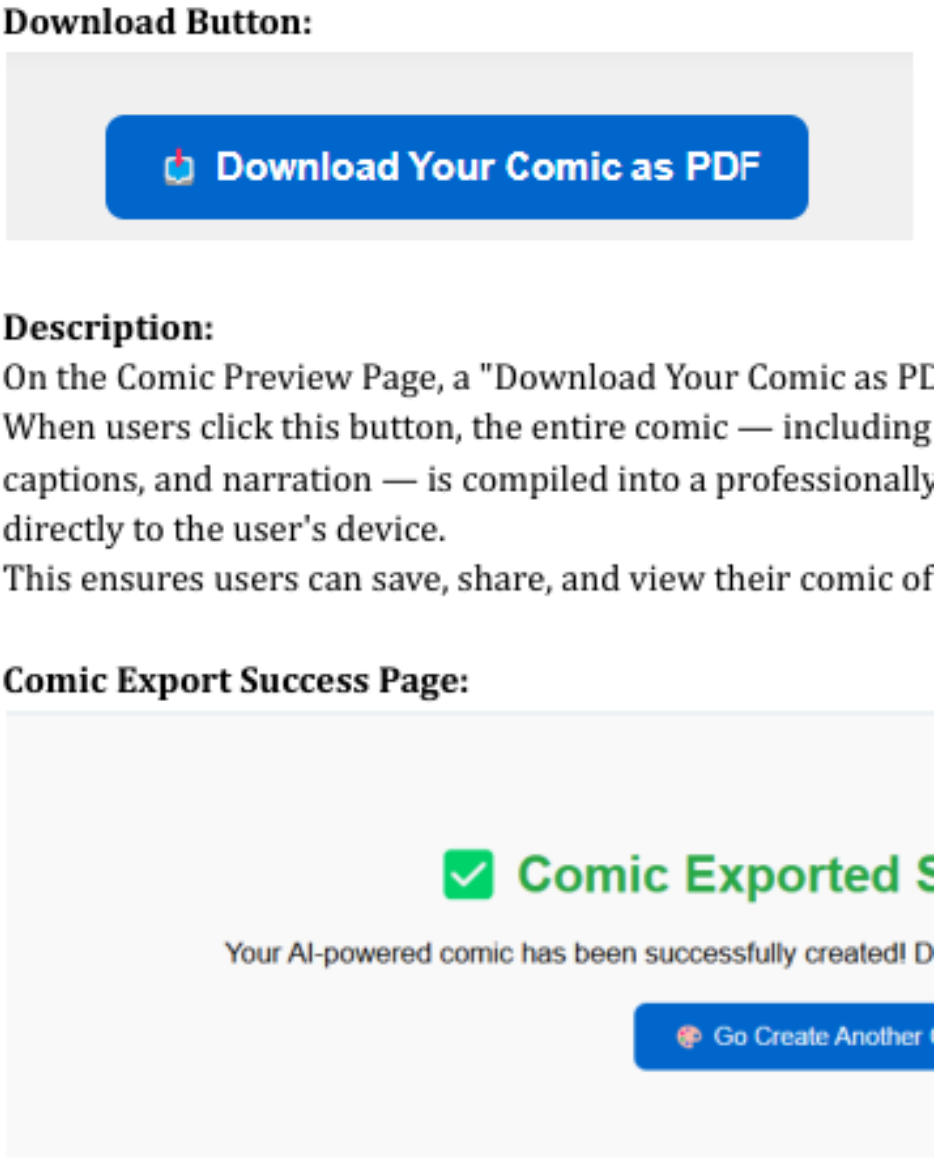
Reference image: a fox comic panel shown in the supplied documentation. The uploaded source's actual demo image generator produces a simple labeled placeholder, unless live image generation is configured.

Panel details example



Reference image: panel title, illustration, scene description, caption, narration, and image prompt example from the supplied reference PDF. The current preview template displays title, image, scene description, caption, narration, and dialogue.

Download and export-success reference



Reference visual showing the PDF download button and export-success message in the supplied PDF. In the uploaded source, the download button is present on comic_preview.html and the success page is reachable at /export-success.

How to capture fresh screenshots

For a final submission that requires screenshots matching the actual running implementation, start the app locally, generate a sample comic in demo mode, and capture the browser pages. Replace these reference visuals with those fresh captures if the live layout differs.

11. Testing and Verification

The project includes tests/test_api.py. The test file sets DEMO_MODE=true and uses FastAPI TestClient to check the homepage, documentation endpoint, JSON generation, form generation, image-test endpoint, and validation behavior.

Included test cases

- test_home — checks that GET / returns HTTP 200.
- test_docs — checks that GET /docs returns HTTP 200.
- test_json — posts a valid JSON request, expects HTTP 200, five panels, and a PDF URL ending in .pdf.
- test_form — posts the HTML form data and expects HTTP 200.
- test_image — checks that /test-image returns success true.
- test_validation — verifies that a too-short story prompt returns HTTP 422.

Run the tests

```
python -m pytest -q
```

The test definitions are present in the uploaded archive. This documentation does not claim a fresh test run was performed during PDF generation. Run the command in your own configured environment to confirm the current installation and dependency compatibility.

Manual smoke test

- Start the server and open http://127.0.0.1:8000.
- Submit the default prompt with DEMO_MODE=true.
- Confirm the preview page contains five panels.
- Click the PDF download link and confirm the file is saved.
- Open /docs and inspect the available endpoints.
- Check static/panels and static/exports for generated output.

12. Configuration and Security

The repository provides .env.example rather than a populated secret file. Copy it to .env for local configuration and keep real credentials out of source control.

Demo mode

DEMO_MODE=true is the supplied default. In this mode the project returns sample story content and creates placeholder panel images. This makes the flow testable without API keys.

Live Gemini text generation

Set DEMO_MODE=false and provide GEMINI_API_KEY to enable calls to the configured Gemini model. The implementation imports google.genai inside the live path. Check the current Google Gen AI SDK documentation and your account's model availability before choosing a model name.

Image generation options

IMAGE_PROVIDER=huggingface requires HF_API_KEY and uses Hugging Face InferenceClient with the configured image model. IMAGE_PROVIDER=diffusers uses a local Diffusers pipeline and requires additional model downloads and compute resources. The demo provider avoids these external calls.

Security checklist

- Never include real API keys in documentation, screenshots, source archives, or public repositories.
- Keep .env excluded from Git; commit .env.example with empty placeholders only.
- Avoid logging credentials or returning secret configuration values in API errors.
- Use appropriate access controls and rate limits before public deployment.
- Review external model provider terms and costs before enabling live generation.

13. Limitations and Future Improvements

Limitations observed in the supplied source

- Demo mode is enabled by default, so generated images are placeholders rather than model-created artwork until configured.
- The layout builder currently returns the panel list unchanged and does not perform additional layout transformation.
- The export-success page exists as a route but is not automatically used after export in the current form flow.
- The route handlers convert any exception into HTTP 500 with the exception text; production error handling should avoid exposing internal details.
- The tests cover key routes but do not test real Gemini/Hugging Face credentials or visual output.
- The PDF exporter uses image paths derived from panel data; validate paths and user-controlled content carefully before public hosting.

Recommended future improvements

- Add structured error pages and user-friendly messages for quota, network, and model failures.
- Add retries, timeout handling, and clearer provider selection/configuration validation.
- Improve PDF layout for oversized images and long narration, and test special characters.
- Add fresh automated UI screenshots and integration tests for actual model responses.

- Add persistence or a comic library so users can revisit prior creations.
- Add deployment instructions for environment variables, file storage, logging, and rate limits.

14. Conclusion

ComicCraft combines a FastAPI web backend, Jinja2 templates, Gemini-compatible story generation, configurable image generation, and FPDF2 export in a modular project structure. Its demo mode supports local workflow checks without API credentials, while the live configuration paths enable external text and image models when valid credentials and dependencies are available.

The supplied code establishes an end-to-end generation pipeline: collect preferences, build panel content, create panel images, display the comic preview, and export a PDF. The report intentionally distinguishes implemented behavior from features described only in the reference PDF. Before final academic submission, run the tests locally and replace the reference visuals with fresh screenshots if an exact match to the current implementation is required.

Appendix: key endpoint list

Endpoint	Expected use
GET /	Open homepage
POST /generate	Submit form and show preview
POST /generate-comic/json	Generate from JSON
POST /test-image	Test image generation
GET /export-success	Open export success page
GET /docs	Explore/test API documentation